

Informe final de accesibilidad WCAG 2.2 AA

Cacao Leaf

Producto web y m?vil para an?lisis preliminar de hojas de cacao mediante im?genes

Versi?n final desplegada para operaci?n inicial

Fecha del informe: 17 de julio de 2026

Documento t?cnico de accesibilidad, trazabilidad funcional y conformidad WCAG 2.2 nivel AA.

Producto evaluado: Cacao Leaf Tipo de producto: plataforma web y m?vil para an?lisis preliminar de hojas de cacao mediante im?genes Versi?n documentada: versi?n final desplegada para operaci?n inicial Fecha del informe: 17 de julio de 2026 Base normativa: Web Content Accessibility Guidelines (WCAG) 2.2, nivel AA Alcance: experiencia web publicada, interfaz m?vil compatible con Expo, API de diagn?stico, flujos de an?lisis, reportes e historial.

1. Resumen ejecutivo

Cacao Leaf es un producto digital orientado a productores, t?cnicos de campo y equipos agr?colas que necesitan registrar evidencias visuales de hojas de cacao, obtener una clasificaci?n preliminar y dar seguimiento a casos observados. El sistema no reemplaza el diagn?stico profesional de un especialista agr?cola; funciona como una herramienta de apoyo para priorizar revisi?n, registrar evidencia y mejorar progresivamente la calidad del an?lisis.

La revisi?n de accesibilidad se realiz? sobre la versi?n final del producto, considerando las ?ltimas actualizaciones implementadas: despliegue en nube, validaci?n de im?genes que no corresponden, reporte de resultados incorrectos, reporte de rechazos, historial con trazabilidad, filtros por tipo de caso y comunicaci?n clara de privacidad y mejora del sistema.

Conclusi?n general: no se identifican incumplimientos cr?ticos desde la revisi?n de c?digo y experiencia funcional. El producto presenta una base accesible y consistente con WCAG 2.2 AA para los flujos principales. Permanecen como validaciones recomendadas las pruebas manuales con lectores de pantalla, navegaci?n por teclado en navegador, orientaci?n horizontal y escalado de texto al 200%.

2. Estado final del producto

2.1 Despliegue y disponibilidad

- ? Frontend web: Netlify, sitio p?blico de Cacao Leaf.
- ? Backend API: Render, servicio cacao-leaf-api.
- ? Repositorio: GitHub, rama principal main.
- ? Backend productivo: <https://cacao-leaf-api.onrender.com>.
- ? Configuraci?n web: netlify.toml apunta el frontend al backend productivo mediante EXPO_PUBLIC_API_URL.

2.2 Funcionalidades principales evaluadas

- ? Captura de imagen desde c?mara.
- ? Selecci?n de imagen desde galer?a.
- ? Env?o de imagen al backend mediante API REST.
- ? Validaci?n de formato, tama?o y contenido visual de la imagen.
- ? Rechazo de im?genes claramente no relacionadas con hojas o vegetaci?n.
- ? Clasificaci?n preliminar como hoja aparentemente sana o posible patolog?a visible.
- ? Resultado con nivel de confianza, observaciones y recomendaci?n.
- ? Reporte de resultado cuando el usuario considera que la clasificaci?n no coincide.
- ? Reporte de rechazo cuando el usuario considera que una imagen rechazada s? era v?lida.
- ? Historial con an?lisis, rechazos reportados, filtros y detalle de cada caso.
- ? Pantalla informativa con limitaciones, uso recomendado y aviso de privacidad/mejora.

2.3 Tecnolog?as del producto

- ? Frontend: React Native, Expo, TypeScript, React Native Web, lucide-react-native.
- ? Backend: Django, Django REST Framework, Gunicorn.
- ? Procesamiento de imagen: Pillow, NumPy, scikit-learn, h5py.

? Persistencia: SQLite en configuraci3n base.

? Despliegue: Netlify para frontend web y Render para API.

3. Alcance de accesibilidad

La revisi3n cubre los flujos de usuario que forman parte de la experiencia final:

1. Inicio y acceso al an3lisis.
2. Captura o carga de imagen.
3. Estado sin imagen.
4. Estado de carga durante an3lisis.
5. Resultado preliminar.
6. Mensajes de error y rechazo de imagen.
7. Reporte de resultado.
8. Reporte de rechazo.
9. Historial con filtros: Todos, Reportados y Rechazos.
10. Detalle de an3lisis y detalle de rechazo.
11. Informaci3n de uso, limitaci3n y privacidad.
12. Navegaci3n inferior.

No se eval3a la precisi3n agron3mica del clasificador como parte de WCAG. La accesibilidad se centra en percepci3n, operaci3n, comprensi3n y compatibilidad t3cnica de la interfaz.

4. Criterios WCAG 2.2 AA considerados

Se revisaron los criterios m3s relevantes para el tipo de producto:

- ? 1.1.1 Contenido no textual: im3genes, iconos y miniaturas.
- ? 1.3.1 Informaci3n y relaciones: estructura de pantallas, encabezados, tarjetas y modales.
- ? 1.3.4 Orientaci3n: uso en orientaci3n vertical y horizontal.
- ? 1.4.1 Uso del color: estados que no dependen exclusivamente del color.
- ? 1.4.3 Contraste m3nimo: contraste de textos principales y estados.
- ? 1.4.4 Cambio de tama3o del texto: compatibilidad con escalado del sistema.
- ? 2.1.1 Teclado: operaci3n en web mediante controles enfocables.
- ? 2.1.2 Sin trampas de teclado: navegaci3n esperada sin bloqueo de foco.
- ? 2.4.3 Orden del foco: lectura y navegaci3n en orden l3gico.
- ? 2.4.6 Encabezados y etiquetas: t3tulos y etiquetas descriptivas.
- ? 2.4.7 Foco visible: estados visuales de controles interactivos.
- ? 2.5.3 Etiqueta en el nombre: coherencia entre texto visible y nombre accesible.
- ? 2.5.8 Tama3o del objetivo: botones y controles t3ctiles de al menos 44 px.
- ? 3.2.1 Al recibir foco: ausencia de cambios inesperados por foco.
- ? 3.3.1 Identificaci3n de errores: errores visibles y anunciabiles.
- ? 3.3.3 Sugerencias ante errores: mensajes que orientan la correcci3n.
- ? 4.1.2 Nombre, funci3n, valor: roles, estados y nombres accesibles.
- ? 4.1.3 Mensajes de estado: carga, errores y confirmaciones.

5. Evaluaci3n por experiencia de usuario

5.1 Inicio

La pantalla de inicio presenta el nombre del producto, una descripción breve y un botón principal para iniciar el análisis. La estructura es simple, sin sobrecarga visual, y utiliza pasos claros para explicar el uso.

Resultado: Cumple. Evidencia: título principal, botón con etiqueta, pasos secuenciales y texto de apoyo.

5.2 Análisis de hoja

La pantalla de análisis permite cargar o galería, muestra el estado sin imagen, conserva una vista previa estable y comunica el procesamiento con un mensaje explícito. El mensaje de carga informa que el servicio puede tardar si estuvo inactivo, lo cual reduce incertidumbre en un despliegue con instancia gratuita.

Resultado: Cumple. Evidencia: botones de acción claros, estado de carga, mensajes de error con `accessibilityRole="alert"`, imagen seleccionada con descripción accesible.

5.3 Resultado preliminar

El resultado muestra estado, confianza, observación y recomendación. No depende solo del color: usa texto, icono, porcentaje y contenido explicativo. Además, permite reportar un resultado si el usuario considera que no coincide.

Resultado: Cumple. Evidencia: estado textual, porcentaje de confianza, notas, recomendación y acción de reporte.

5.4 Rechazo de imagen

Cuando la imagen no parece corresponder a una hoja o vegetación, el sistema muestra un mensaje accionable y permite reportar el rechazo. Esto evita que un posible falso rechazo quede sin trazabilidad.

Resultado: Cumple. Evidencia: mensaje específico, acción Reportar rechazo, formulario con motivo y comentario opcional.

5.5 Historial y trazabilidad

El historial funciona como centro de revisión. Integra análisis normales, resultados reportados y rechazos reportados. Incluye filtros con contadores: Todos, Reportados y Rechazos. El detalle permite revisar evidencia, fecha, resultado, comentario y último reporte.

Resultado: Cumple. Evidencia: filtros, contadores, etiquetas de estado, detalle modal, acción de reporte desde historial.

5.6 Información, limitaciones y privacidad

La pantalla informativa comunica patologías visibles, limitaciones del análisis, uso recomendado y uso de imágenes reportadas para revisión técnica y mejora del sistema.

Resultado: Cumple. Evidencia: bloques temáticos con encabezados, texto claro y aviso de privacidad/mejora.

6. Resultados por criterio

Código	Criterio WCAG	Resultado	Evidencia principal	Riesgo residual
AX-01	1.1.1 Contenido no textual	Cumple	Imágenes y miniaturas tienen etiquetas; iconos decorativos se ocultan del lector	Validar lectura real con lector de pantalla
AX-02	1.3.1 Información y relaciones	Cumple	Encabezados, bloques, modales y filas siguen jerarquía lógica	Validación manual de orden de lectura
AX-03	1.3.4 Orientación	Cumple	Expo configurado con <code>orientation: default</code>	Revisar visualmente en landscape
AX-04	1.4.1 Uso del color	Cumple	Estados usan color, texto, iconos y etiquetas	Mantener si se agregan nuevos estados
AX-05	1.4.3 Contraste mínimo	Cumple	Paleta principal supera contraste AA en textos críticos	Recalcular si cambia la paleta

Código	Criterio WCAG	Resultado	Evidencia principal	Riesgo residual
AX-06	1.4.4 Tamaño del texto	Cumple parcialmente	No se desactiva escalado de fuente	Probar texto al 200%
AX-07	2.1.1 Teclado	Cumple parcialmente	Controles interactivos basados en Pressable	Probar Tab, Enter, Espacio en web
AX-08	2.1.2 Sin trampas de teclado	Cumple parcialmente	No hay lógica que capture foco permanentemente	Probar modales en navegador
AX-09	2.4.3 Orden del foco	Cumple parcialmente	Orden visual y DOM siguen flujo lógico	Confirmar foco inicial y retorno
AX-10	2.4.6 Encabezados y etiquetas	Cumple	Títulos, botones, acciones y filtros son descriptivos	Mantener consistencia en futuras pantallas
AX-11	2.4.7 Foco visible	Cumple parcialmente	Estados visuales de controles presentes	Verificar foco en web con teclado
AX-12	2.5.3 Etiqueta en el nombre	Cumple	Texto visible coincide con nombres accesibles	Revisar nuevos botones futuros
AX-13	2.5.8 Tamaño del objetivo	Cumple	Botones principales y cierres cumplen mínimo táctil	Mantener mínimo de 44 px
AX-14	3.3.1 Identificación de errores	Cumple	Errores visibles y anunciados	Validar anuncio con lector
AX-15	3.3.3 Sugerencias ante errores	Cumple	Mensajes indican formato, tamaño, contenido o permisos	Mantener mensajes específicos
AX-16	4.1.2 Nombre, función, valor	Cumple	Roles, estados, labels e hints en controles	Prueba manual con lector
AX-17	4.1.3 Mensajes de estado	Cumple	Carga, error y confirmación usan estados visibles	Validar live region en navegador/dispositivo

7. Hallazgos y mejoras aplicadas

7.1 Mejoras en comunicación de estado

Se reforzó el estado de carga durante el análisis con el mensaje: "Analizando imagen. Si el servicio estuvo inactivo, puede tardar unos segundos." Esto es relevante para una API desplegada en Render con instancia gratuita, donde el primer request puede demorar.

7.2 Mejora de trazabilidad operativa

El historial dejó de ser solo una lista de análisis. Ahora funciona como herramienta de revisión con filtros y contadores, diferenciando casos normales, reportados y rechazos.

7.3 Mejora de reportes

El producto permite reportar dos tipos de situaciones:

- Resultado generado que el usuario considera incorrecto.
- Imagen rechazada que el usuario considera válida.

Ambos casos se conservan como evidencia para revisión y mejora futura.

7.4 Mejora de textos y tono profesional

Se corrigió el tono de interfaz para producto final: textos con acentos, etiquetas claras, advertencias explícitas y comunicación de limitaciones sin tratar la app como prototipo académico.

7.5 Privacidad y mejora

Se agregó un bloque informativo que comunica que las imágenes y observaciones reportadas pueden conservarse para revisión técnica y mejora del sistema.

8. Verificaciones técnicas realizadas

Durante el cierre del producto se ejecutaron verificaciones técnicas sobre frontend y backend:

- ? python manage.py check: correcto.
- ? python manage.py test: correcto, 12 pruebas automatizadas.
- ? npm run build:web: correcto, exportación web generada para Netlify.

Estas verificaciones no reemplazan una auditoría manual con tecnologías asistivas, pero reducen el riesgo de errores funcionales en la entrega.

9. Riesgos residuales

Los siguientes puntos quedan identificados como controles manuales recomendados antes de una certificación formal:

1. Probar TalkBack en Android.
2. Probar VoiceOver en iOS.
3. Verificar navegación por teclado en navegador: Tab, Shift+Tab, Enter y Espacio.
4. Confirmar foco inicial y retorno del foco en modales.
5. Probar escalado de texto al 200%.
6. Probar orientación horizontal en pantallas principales.
7. Ejecutar auditoría automatizada con Lighthouse o axe sobre la versión web publicada.

Ninguno de estos puntos representa una barrera crítica detectada desde el código; son validaciones de conformidad manual esperadas en un proceso formal WCAG.

10. Checklist final recomendado

Prueba	Resultado esperado
Abrir el sitio web publicado	La app carga sin errores y permite iniciar análisis
Usar cámara o galería	El usuario puede seleccionar una imagen
Analizar hoja válida	Se muestra resultado, confianza, observación y recomendación
Enviar imagen no relacionada	Se muestra rechazo con explicación y opción de reporte
Reportar resultado	El caso queda marcado como reportado
Reportar rechazo	El rechazo aparece en historial
Revisar historial	Se muestran filtros con contadores
Abrir detalle	El modal muestra datos del caso y permite cerrar
Consultar información	Se muestran limitaciones, uso recomendado y privacidad
Navegar con teclado en web	Los controles principales son alcanzables y operables
Usar lector de pantalla	Títulos, botones, estados y errores son anunciados correctamente

11. Dictamen final

Cacao Leaf presenta una experiencia alineada con WCAG 2.2 AA para los flujos principales de uso. La interfaz es operable, comprensible y consistente; los errores son visibles y accionables; los resultados no dependen exclusivamente del color; los controles principales tienen tamaño táctil adecuado; y el historial conserva trazabilidad de casos observados.

El producto se considera listo para entrega final y operaci3n inicial, con la recomendaci3n de ejecutar pruebas manuales con lectores de pantalla y teclado antes de declarar conformidad formal completa.

12. Archivos relacionados

- ? mobile/src/screens/HomeScreen.tsx
- ? mobile/src/screens/AnalyzeScreen.tsx
- ? mobile/src/screens/HistoryScreen.tsx
- ? mobile/src/screens/InfoScreen.tsx
- ? mobile/src/components/PrimaryButton.tsx
- ? mobile/src/components/TabBar.tsx
- ? mobile/src/theme/colors.ts
- ? mobile/src/api/client.ts
- ? backend/diagnostics/serializers.py
- ? backend/diagnostics/views.py
- ? netlify.toml
- ? render.yaml